



1. Problem Title & Link

Title: LeetCode 1275 – Find Winner on a Tic Tac Toe Game

Link: <https://leetcode.com/problems/find-winner-on-a-tic-tac-toe-game/>

2. Problem Statement (Short Summary)

Two players, **A** and **B**, play Tic Tac Toe on a **3 × 3** board.

- Player A always moves first.
- The input is a list of moves.
- Determine whether:
 - A wins,
 - B wins,
 - the game is a Draw,
 - or the game is still Pending.

Return "A", "B", "Draw", or "Pending".

3. Examples (Input → Output)

Example 1

Input

moves = [[0,0],[2,0],[1,1],[2,1],[2,2]]

Output

"A"

Explanation

A | |

| A |

B | B | A

A completes the main diagonal.

Example 2

Input

moves = [[0,0],[1,1],[0,1],[0,2],[1,0],[2,0]]

Output

"B"

Example 3

Input



moves = [[0,0],[1,1]]

Output

"Pending"

4. Constraints

- $1 \leq \text{moves.length} \leq 9$
- Board size is always 3×3
- No duplicate moves
- Every move is valid

5. Core Concept (Pattern / Topic)

- Matrix Simulation
- Counting
- Game Logic

6. Thought Process (Step-by-Step Explanation)

Brute Force

Create a 3×3 board.

After every move:

- Fill the board.
- Check every row.
- Check every column.
- Check both diagonals.

Time Complexity:

$O(9 \times 8)$

Since the board is tiny, this works.

Optimized Thinking

Instead of checking the entire board repeatedly, maintain counts for:

- Each row
- Each column
- Main diagonal
- Anti-diagonal

Represent

- Player A as **+1**
- Player B as **-1**



Whenever a move is played,
update the corresponding counts.

If any count becomes

3

Player A wins.

If any count becomes

-3

Player B wins.

Intuition

Each move contributes

A -> +1

B -> -1

Example

Rows

0 : +1

1 : +1

2 : +1

Row total = 3

↓

A wins.

Why This Works

A player wins only when all three cells in a row, column, or diagonal belong to the same player.

Using +1 and -1 automatically tracks ownership.

7. Visual / Intuition Diagram (ASCII)

Example

Move 1

A

X . .

. . .

. . .

Rows = [1, 0, 0]



Move 2

```
O . .  
. X .  
. . .
```

Diagonal = 2

Move 3

```
X . .  
. X .  
. . X
```

Diagonal = 3

↓

A Wins

8. Pseudocode (Language Independent)

```
rows[3]  
cols[3]  
diag  
antiDiag  
  
for every move  
    if player A  
        value = 1  
    else  
        value = -1  
  
    update row  
    update column  
  
    if main diagonal  
        update diag  
  
    if anti diagonal  
        update antiDiag  
  
    if abs(any count) == 3  
        return winner
```



```
if moves == 9
    return Draw

return Pending
```

9. Code Implementation

Python

```
class Solution:
    def tictactoe(self, moves: list[list[int]]) -> str:

        rows = [0] * 3
        cols = [0] * 3
        diag = 0
        anti = 0

        for i, (r, c) in enumerate(moves):

            player = 1 if i % 2 == 0 else -1

            rows[r] += player
            cols[c] += player

            if r == c:
                diag += player

            if r + c == 2:
                anti += player

            if (
                abs(rows[r]) == 3 or
                abs(cols[c]) == 3 or
                abs(diag) == 3 or
                abs(anti) == 3
            ):
                return "A" if player == 1 else "B"

        return "Draw" if len(moves) == 9 else "Pending"
```

Java

```
class Solution {

    public String tictactoe(int[][] moves) {

        int[] rows = new int[3];
        int[] cols = new int[3];

        int diag = 0;
```



```
int anti = 0;

for (int i = 0; i < moves.length; i++) {

    int r = moves[i][0];
    int c = moves[i][1];

    int player = (i % 2 == 0) ? 1 : -1;

    rows[r] += player;
    cols[c] += player;

    if (r == c)
        diag += player;

    if (r + c == 2)
        anti += player;

    if (Math.abs(rows[r]) == 3 ||
        Math.abs(cols[c]) == 3 ||
        Math.abs(diag) == 3 ||
        Math.abs(anti) == 3) {

        return player == 1 ? "A" : "B";
    }
}

return moves.length == 9 ? "Draw" : "Pending";
}
```

10. Time & Space Complexity

Time Complexity

$O(n)$

- n = number of moves
- Maximum moves = 9

Practically,

$O(1)$

Space Complexity

$O(1)$

Only four counters and two arrays of size 3 are used.

11. Common Mistakes / Edge Cases

- Forgetting the anti-diagonal condition ($r + c == 2$)



- Mixing Player A and Player B turns
- Returning "Draw" before checking for a winner
- Forgetting "Pending" when fewer than 9 moves exist
- Updating the wrong row or column

12. Detailed Dry Run

Example

moves =

[[0,0],

[2,0],

[1,1],

[2,1],

[2,2]]

Mov e	Playe r	Positio n	Ro w	Co l	Dia g	Ant i	Winne r
1	A	(0,0)	1	1	1	0	No
2	B	(2,0)	-1	0	1	-1	No
3	A	(1,1)	1	1	2	1	No
4	B	(2,1)	-2	0	2	1	No
5	A	(2,2)	-1	1	3	1	A

Final Output

"A"

13. Common Use Cases (Real-Life / Interview)

- Tic Tac Toe engines
- Turn-based game simulation
- Board state evaluation
- Score tracking systems
- Event counting problems

14. Common Traps (Important!)

- Forgetting to alternate players
- Incorrect anti-diagonal formula
- Using separate board scanning after every move
- Returning Draw before checking the last move
- Updating counters after winner detection



15. Builds To (Related LeetCode Problems)

- LC 348 – Design Tic-Tac-Toe
- LC 36 – Valid Sudoku
- LC 37 – Sudoku Solver
- LC 794 – Valid Tic-Tac-Toe State
- LC 289 – Game of Life

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Board Simulation	$O(9^2)$	$O(9)$	Easy to understand
Check board after every move	$O(9 \times 8)$	$O(9)$	Repeated work
Row/Column Counters	$O(n)$	$O(1)$	Best interview solution

17. Why This Solution Works (Short Intuition)

Each move updates only one row, one column, and possibly one or two diagonals. Using +1 for Player A and -1 for Player B, a count reaching 3 or -3 means all three cells in that line belong to the same player, allowing us to detect the winner immediately without scanning the board.

18. Variations / Follow-Up Questions

1. Extend the solution for an $N \times N$ Tic Tac Toe board.
2. Design a class that supports moves one at a time (similar to LC 348).
3. Detect multiple winning lines.
4. Validate whether a board state is possible.
5. Generalize the solution for Connect Four or Gomoku.
- 6.